

AI Tools: Introduction to Claude Code

Xugan Chen

Yale School of Management

MAM Orientation BootCamp

August 27, 2026

github.com/xuganchen/BootCamp-Introduction-to-Claude-Code

About me

- ▶ **Xugan Chen** – PhD candidate, Financial Economics, Yale SOM. At Yale since 2020 (predoc 2020–2022, PhD since 2022).
- ▶ **TA for:** Entrepreneurial Finance; Venture Capital and Private Equity; Statistical Foundations; Behavioral Finance.
- ▶ **Research:** innovation; AI in finance and economics; corporate finance; entrepreneurial finance.

Feedback is welcome – now, or any time after today.

Getting to know you

Show of hands. Which side of each pair are you here for?

Public markets vs **Private markets**

Industry vs **Academia**

Quant research vs **Fundamental research**

Equities vs **Other asset classes**

Today's task sits in **private markets**, and the workflow is the same whichever side you raised your hand on.

Getting to know you: How do you use AI tools?

Tool	What you actually do
Browser chat	Copy code out, paste it in, run it, paste the error back
Agentic IDE (Cursor)	Tell it what to do; it edits files inside the editor
Terminal agent (Claude Code, Codex)	Full environment: reads, writes, runs commands, plans
Terminal agent plus MCP tools	Wire it to databases, APIs, other services
Agent in a container	One instruction, and leave alone

Today is the terminal agent, with one look at agents in a container at the end. Agent in a container is not better; it is more setup.

Ladder adapted from Kyle Jensen.

Working with AI, one year ago

- ▶ A **chat window**. We describe the problem in prose, it hands back a snippet, we paste the snippet.
- ▶ It could not see our data, our files, or our errors. We were the clipboard between the model and the project.
- ▶ The code was **roughly right and often not runnable**, which meant we still had to be the one who ran it.

That is the version most people still have in mind.

Working with AI, now: it acts in our project

An agent that lives in our terminal, inside our project directory.

- ▶ **Reads and writes files** in the repo – our data, our code, our notes.
- ▶ **Runs commands** – installs packages, executes scripts, reads the errors.
- ▶ **Iterates** – sees its own output fail and tries again, without us retyping.

It does not hand us a snippet to paste. It acts in the project and **lives with the consequences.**

The loop is easy. Instruct and review is the job.

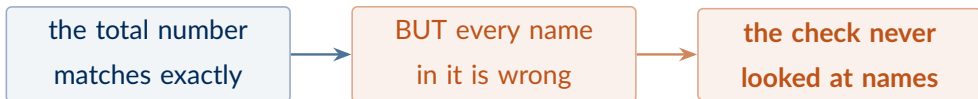


- ▶ The two boxes with our names on them are **the entire job**.
- ▶ Everything we add today – planning, verification, generalization – makes the instruction sharper, or makes review possible on output that is **too large to read**.

For example, today we will sign off on a dataset of 37,197 rows without reading one of them.

Why reviewing is hard: it checks its own work

- ▶ Ask it to build a dataset and you usually get a check you did not ask for: “I reconciled the total to the reported figure – it matches.”
- ▶ That is genuinely useful. It is also the trap, because **a check looks at one thing**, and the agent decided which thing.



A check that passes tells you about **the part it looked at**. It says nothing about the rest.

We are going to meet this four times this afternoon, and it looks different every time.

Today's objective

- ▶ **Not** a feature walkthrough – the menu of options will have changed by October.
- ▶ **Not** an intro to programming – most of you can already code.
- ▶ **Is yours to build**, with the agent, on your own machine – not a demo you watch.
- ▶ **Is** practice at **checking output you cannot read** – tens of thousands of rows.
- ▶ **Is** practice at **patterns that move** to the rest of your work, not just to this task.

It is never “**can it write the code**”. It is “**how do I know this is right**” and “**how do I scale it**”.

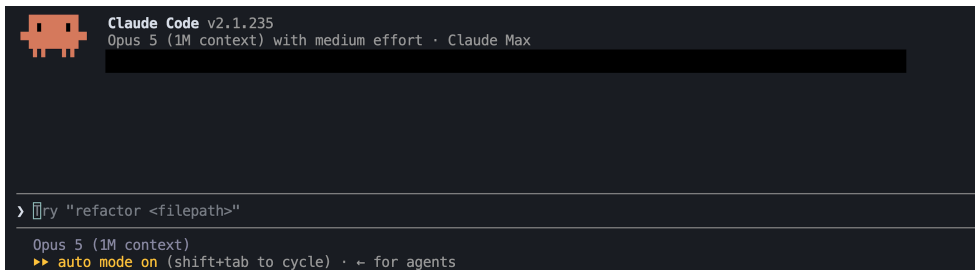
The roadmap

Block	Mode
1. Introduction	lecture
2. Applied AM project: a private credit dataset from SEC filings	demo, then hands-on
Break , 10 minutes – halfway through the project	
... <i>the project continues</i> : generalize, then analyse	hands-on
3. More useful concepts and tips	lecture
4. Advanced usage examples	demo

Half the session (in Part 2) is hands-on. So have your laptop out and Claude Code installed. Work on your own or with the person next to you – either is fine.

Basic setup

- ▶ **Install** it, either way:
 - ▶ Antigravity CLI: Claim the student plan in gemini.google/students/ and install it from antigravity.google/download
 - ▶ Claude Code: Subscribe to a paid plan and install it from code.claude.com/docs
- ▶ **Then** `cd` into a project folder and type `claude` (or `agy`). That is all.

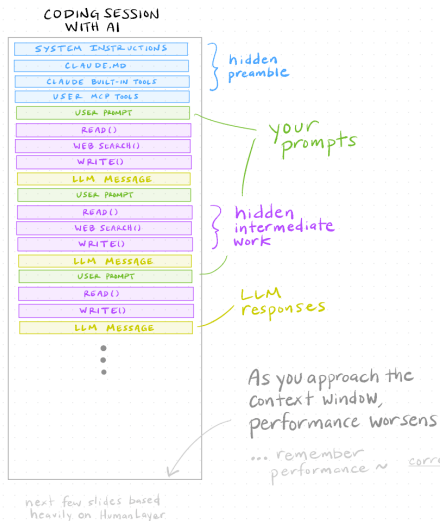


If your terminal doesn't look like this, fix it now. The rest of the session assumes a working install.

Tokens: the unit everything is measured in

- ▶ **Tokens** are the pieces text is chopped into before the model sees it.
 - ▶ The short sentence “I love cats” is broken into **3 tokens** (I, love, cats).
 - ▶ A longer word like “unbreakable” may be split into **2 tokens** (un and breakable).
 - ▶ Rule of thumb: in English, **1 token is about 4 characters, or 0.75 words**.
- ▶ Claude Fable/Opus/Sonnet: a **1M-token context window**, roughly 750,000 words.
- ▶ Later we will work on SEC filings – **one filing runs 10 to 25 MB of HTML**.
 - ▶ At 25 MB that is roughly **6 million tokens – six full context windows**.
 - ▶ **So nothing ever reads the whole document**. It gets searched, sliced, and read a slice at a time. You will watch that happen this afternoon.

Context window: Long sessions degrade, so plan for it.



- ▶ A “chat” is **one long document** passed back and forth: system prompt, tool definitions, thinking, tool calls, results. All of it counts against the window, and we only see the output.
- ▶ **Write state to files.** A decision that lives only in the conversation is gone. Files persist.
- ▶ **Work in chunks.** Five to ten turns with one objective, then start fresh. Your agent at turn 30 is noticeably worse than at turn 3.
- ▶ **Compact.** /compact summarizes, writes to files, and continues.

Source: A Series on Claude Code by Paul Goldsmith-Pinkham.

Housekeeping

- ▶ **No homework, no grading.** Nothing is submitted.
- ▶ **Everything ships:** every prompt as it was run, every plan – and the failures.
- ▶ **No code is provided** – We will build everything with the agent.

`github.com/xuganchen/BootCamp-Introduction-to-Claude-Code`

Part 2. Building a private credit dataset from raw SEC filings

Why private credit, and why BDCs

- ▶ Private credit is the **fastest-growing corner of asset management**.
- ▶ **BDC filings are its only public window**. A BDC lends to middle-market borrowers and must disclose every position, every quarter.
- ▶ **The filing grades your work**. The positions foot to the balance sheet, so you can check the agent against an audited number.

A real AM dataset, with a ground truth to check it against, free and public.

Investment position data from raw SEC filings

- ▶ A position-by-position table of **every loan the fund holds**: borrower, industry, lien seniority, spread, maturity, cost, fair value.
- ▶ Filed quarterly inside the 10-K and 10-Q, back roughly two decades.
- ▶ Example – ARCC's 2026 Q2 10-Q: **24.8 MB, 275 HTML tables, ~6 million tokens**.

You cannot paste this into a chat window.

Company (1)	Business Description	Investment (18)	Coupon (3)	Reference (7)	Spread (3)	Acquisition Date	Maturity Date	Shares/Units	Principal	Amortized Cost	Fair Value
Software and Services											
ACP Avenu Midco LLC (13)	Provider of technology solutions and hardware to U.S. state and local governments and motor vehicle agencies	First lien senior secured revolving loan	8.65%	SOFR (Q)	5.00%	04/2025	10/2029		\$ 0.9	\$ 0.9	\$ 0.9
		First lien senior secured loan	8.69%	SOFR (Q)	5.00%	08/2025	10/2029		3.6	3.5	3.6
		First lien senior secured loan	8.69%	SOFR (Q)	5.00%	04/2025	10/2029		13.1	13.0	13.1
										17.4	17.6
Actfy Buyer, Inc. (13)	Software provider of end to end fraud management workflow solutions	First lien senior secured loan	8.39%	SOFR (M)	4.75%	05/2024	05/2031		2.1	2.1	2.1
Activate Holdings (US) Corp. and CrossPoint Capital AS SPV, LP (13)	Provider of software services that support the management and security of computing devices, applications, data, and networks	First lien senior secured loan	8.98%	SOFR (Q)	5.25%	09/2024	07/2030		6.8	6.8	6.8
		First lien senior secured loan	8.98%	SOFR (Q)	5.25%	07/2023	07/2030		41.9	41.9	41.9
		Limited partnership interest				10/2023		9,249,000		10.3	13.2
Adonis Bidco Inc. (13)	Provider of intellectual property management lifecycle software									59.0	61.9
		First lien senior secured loan	9.23%	SOFR (Q)	5.50%	02/2025	02/2032		30.0	30.0	28.5
		First lien senior secured loan	9.48% (2.88% PIK)	SOFR (Q)	5.75%	02/2025	02/2032		238.6	238.6	226.7
										268.6	255.2

Building a private credit dataset from SEC filings

▶ Panel A – BDC-quarter

- ▶ One row per BDC per quarter: **fund-level financial statements**.
- ▶ NAV, leverage, portfolio yield, portfolio size, non-accruals, etc.

▶ Panel B – BDC-quarter-investment

- ▶ One row per investment position per quarter: **deal-level terms**.
- ▶ Borrower, loan type, seniority, spread, maturity, cost, fair value, etc.

▶ Pick your own BDC (a large one is easiest), window **2023-01-01 to now**.

Step	What we build	The concept it forces
0	Naive baseline: one prompt	A passing check is only as good as its scope
1	Write a spec, then build	Plan mode, sub-agents for independent verification
2	Generalize across quarters	Skills , and the tie-out as a test suite
3	Analysis and report	Analysis as verification; grounding in your references

Step 0. The naive baseline

Step	What we build	The concept it forces
0	Naive baseline: one prompt	A passing check is only as good as its scope
1	Write a spec, then build	Plan mode , sub-agents for independent verification
2	Generalize across quarters	Skills , and the tie-out as a test suite
3	Analysis and report	Analysis as verification; grounding in your references

- ▶ I will walk through my own output in detail; you do not need to type anything.
- ▶ It took ~15 to 20 minutes to finish this one simple prompt.
- ▶ I experimented with three different models: Opus 5 at xhigh effort, Opus 5 at medium effort, and Sonnet 5 at medium effort.

Start with the laziest possible prompt

```
Find a large publicly traded BDC, download its most recent 10-Q from SEC EDGAR, and parse  
the financial statements and the schedule of investments into two structural tables I  
can analyze.
```

- ▶ Not in it: a ticker, a schema, a file format, a variable list.
- ▶ Not in it: how to download or paste the filing.
- ▶ Not in it: any word about verifying anything.

It picked its own target, and looked before it parsed

```
Claude Code v2.1.233
Welcome back Xugan!



Tips for getting started
Run /init to create a CLAUDE.md file with instructions for ...

What's new
Added GitHub merge request URL support to the '--worktree' ...
Added an opt-in 'forward_user_identity' apps gateway settin...
Added opt-in memory cgroup support for Bash tool commands o...
/release-notes for more

1 MCP server needs authentication · run /mcp
> /rename bdc-parse@
  ↳ Session renamed to: bdc-parse@

> Find a large publicly traded BDC, download its most recent 10-Q from SEC EDGAR, and parse the financial statements
and the schedule of investments into two structural tables I can analyze.

• I'll start by looking at the working directory, then go to EDGAR.
  Listed 1 directory, ran 1 shell command

• Empty directory, clean start. Let me pull the filing index from EDGAR.
  Ran 1 shell command

• ARCC (Ares Capital) – largest publicly traded BDC. Most recent 10-Q: period 2026-06-30, filed 2026-07-29.
  Downloading it.
  Ran 1 shell command

• 25 MB. Let me survey its structure before parsing.
  Ran 1 shell command
```

```
• SOI is XBRL-tagged (R9). Let me pull the rendered statement files.
  Ran 2 shell commands

• R9's rendering stacks label/value rows – awkward. The inline XBRL in the primary document is the better source. Let
me survey its facts and dimensions.
  Ran 4 shell commands

• R9 is only the issuer-level rollup (cost/fair value). The per-tranche detail lives in the primary document's SOI
tables. Let me locate them.
  Ran 2 shell commands

• Now I understand the SOI layout. Let me inspect the inline XBRL markup inside SOI rows to build a
position-independent parser.
  Ran 1 shell command

• Very clean – each investment line has its own context, with scale on the numerics. Let me check the context
definitions.
  Ran 1 shell command

• Contexts confirm 1,954 investment lines at 2026-06-30 and 1,906 at 2025-12-31, plus issuer subtotals. Let me verify
column alignment via cumulative colspan.
  Ran 1 shell command

• Column grid is stable at 75 columns. Let me identify the full extent of the SOI tables and catalogue row types.
  Ran 1 shell command

• SOI spans tables 9-75 (Jun 30, 2026) and 90-154 (Dec 31, 2025). Let me catalogue the non-investment row types.
  Ran 2 shell commands
```

- ▶ **Ares Capital (ARCC)**, reasoned from “large publicly traded BDC”, and its **most recent 10-Q** (2026-06-30): a 24.8 MB inline-XBRL document.
- ▶ It then **surveyed the filing** before parsing – how many tables, where the schedule starts, how the columns repeat.
- ▶ “**Look before you act**” costs a couple of minutes, and it is why the parse works.

What came back, from one sentence

- Done. Both tables are built and reconcile to the filing.

Result

- BDC: Ares Capital Corporation (ARCC), largest publicly traded BDC, \$30.5bn total assets at 2026-06-30 (sources: parsed balance sheet). Form 10-Q for Q2 2026, filed 2026-07-29, accession 0001628280-26-050307, pulled from SEC EDGAR.
- Two tables: output/financial_statements.csv (435 facts, all four statements) and output/soi_investments.csv (2,847 investment lines x 44 columns).
- Verified: the sum of parsed investment lines equals the filing's printed "Total Investments" exactly, at both dates, for both amortized cost and fair value.

Files

path	contents
data/raw/arcc-20260630.htm	24.8 MB source filing
code/parsers/arcc_10q.py	the parser, 800 lines, documented
output/financial_statements.csv	tidy long, one row per reported fact
output/soi_investments.csv	one row per investment line
output/soi_subtotals.csv	issuer/industry/grand subtotals the filing prints
output/soi_footnote_legend.csv	both 10-note legends
output/tieout_report.txt	verification, regenerated each run
output/README.md	data dictionary

- ▶ It took ~22 minutes.
- ▶ Two tables: 435 statement facts; **2,847** investment rows (1,439 at 2026-06-30, 1,408 at 2025-12-31).
- ▶ Plus a footnote legend, a subtotals file, a README, and a **re-runnable Python parser** – 803 lines of it.

948.00% coupon – and the filing is wrong, not the agent

```
• One clear bug: Adonis Bidco shows coupon 948.00%. Let me inspect that row's markup.

import pandas as pd
invmpd.read_csv('output/soi_investments.csv')
for c in ['coupon_pct', 'pik_pct', 'spread_pct', 'pct_of_net_assets', 'pct_of_shares_owned']:
    s=inv[c].dropna()
    bad=inv[inv[c]>100]
    print(f'{c:22s} {(len(s):5d) min={s.min():9.2f} max={s.max():12.2f} >100: {len(bad)}}')
print(inv[inv.coupon_pct>100][['period_end', 'portfolio_company', 'investment_type', 'coupon_pct', 'pik_pct', 'spread_pct', 'source_table', 'context_id']].head(12).to_string(index=False))
PY)
┌ coupon_pct      n= 1846 min= 3.61 max= 991.00 >100: 3
┌ pik_pct         n= 369 min= 0.25 max= 288.00 >100: 2
┌ spread_pct      n= 1838 min= 0.75 max= 12.15 >100: 0
┌ pct_of_net_assets n= 0 min= nan max= nan >100: 0
┌ pct_of_shares_owned n= 30 min= 0.42 max= 100.00 >100: 0

period_end      portfolio_company      investment_type      coupon_pct      pik_pct
spread_pct      source_table      context_id
2026-06-30      9      Adonis Bidco Inc.      First lien senior secured loan      948.0      288.0
5.75
2026-06-30      Covert HoldCo, LP      Senior subordinated loan      948.0      NaN
5.75      34      c-725
2026-06-30      Mountaineer Merger Corporation First lien senior secured revolving loan      991.0      125.0
6.25      58      c-1394

from lxml import etree
p = etree.HTMLParser(huge_tree=True)
root = etree.parse('data/raw/arcc-20260630.htm', p).getroot()
tables=root.xpath('//table')
def loc(e): return (e.tag if isinstance(e.tag, str) else '', split('')[-1].split('')[-1].lower())
def tx(e): return ' '.join(' '.join(e.iter_text()).replace('\x0d', '').split())
for r in tables[9].xpath('//tr'):
    if any([x.get('contextref')=='c-36' for x in r.iter() if loc(x)=='nonfraction']):
        print('ROW TEXT:', tx(r)[200:], '\n')
        for c in r.xpath('//td'):
            ix=id for d in c.iter() if loc(d)=='nonfraction'
            if ix:
                print('CELL:', repr(tx(c)))
                for d in ix:
                    print(' ', d.get('name'), 'scale=', d.get('scale'), 'unit=', d.get('unitref'),
'dec=', d.get('decimals'), 'text=', repr(tx(d)))
                break
PY)
┌ ROW TEXT: First lien senior secured loan9.48% (2.88% PIK)SOFR (Q)5.75%02/202502/2832238.6 238.6 226.7 (2)(9)
CELL: '9.48% (2.88% PIK)'
us-gaap:InvestmentInterestRate scale= 0 unit= rate dec= INF text= '9.48'
us-gaap:InvestmentInterestRatePaidInKind scale= 0 unit= rate dec= INF text= '2.88'
CELL: '5.75%'
us-gaap:InvestmentBasisSpreadVariableRate scale= -2 unit= rate dec= INF text= '5.75'
CELL: '238.6'
us-gaap:InvestmentOwnedBalancePrincipalAmount scale= 6 unit= usd dec= -5 text= '238.6'
CELL: '238.6'
us-gaap:InvestmentOwnedAtCost scale= 6 unit= usd dec= -5 text= '238.6'
CELL: '226.7'
us-gaap:InvestmentOwnedAtFairValue scale= 6 unit= usd dec= -5 text= '226.7'

• A real tagging error in the filing: those 3 rows carry scale="0" on the rate while every other row uses scale="-2".
The printed value is 9.48%. Let me take rates as printed.
```

- ▶ It also found an obvious bug: Adonis Bidco shows a coupon of 948.00%.
- ▶ Then it dug in: three of the 1,846 coupon facts (Adonis Bidco, Covert HoldCo, Mountaineer Merger) are tagged scale="0" where every other rate carries scale="-2". Taking the XBRL value at face value turns a 9.48% coupon into 948%.
- ▶ The mis-tagging is ARCC's, sitting live at SEC.
- ▶ The agent noticed it, worked around it, and wrote it down.

Nobody asked for verification. It built one.

Check, each against a number the filing prints	2026-06-30	2025-12-31
A. parsed lines vs printed Total Investments	exact	exact
B. each issuer subtotal vs the lines above it	394/394 agree	380/380 agree
C. schedule total vs balance sheet	within \$0.4m	within \$0.2m
D. affiliation split vs the three balance sheet buckets	within \$0.8m	within \$0.8m
E. investments / equity vs the printed % of net assets	211.28 vs 211.32	205.93 vs 205.93

We didn't ask for verification, but it invented a balance-sheet tie-out anyway.

If we stopped here, we would think the lesson was “the prompt can be lazy”.

Same prompt, three times

- ▶ Identical prompt, identical filing, three models:
 - ▶ Opus 5 at xhigh effort
 - ▶ Opus 5 at medium effort (max > xhigh > high > medium > low)
 - ▶ Sonnet 5 at medium effort (Fable 5 > Opus 5 > Sonnet 5)
- ▶ Nothing else differs. No extra instruction, no follow-up, no correction.
- ▶ This is the experiment sixty of you run every time you all point the same tool at the same task: **same instruction, different machine and model.**

Question: what do you expect to differ – and what do you expect to stay the same?

Three right answers with no shared vocabulary

All three chose ARCC, parsed both periods, and passed a self-invented tie-out at 0.001%.

Every run succeeded. Then:

	Opus 5 xhigh	Opus 5 medium	Sonnet 5 medium
Columns	44	22	26
Company field	portfolio_company	issuer	issuer
Fair value field	fair_value_usd	fair_value	fair_value_usd
Type field	investment_type	instrument	investment_type
Verification shipped as	tieout_report.txt	separate script	inside the parser
Parser	803 lines, 1 file	526 lines, 2 files	504 lines, 2 files

There are sixty of you in this room. That is sixty correct, but incompatible datasets.

Count the rows. What does the 1,409 mean?

Run	date	rows	distinct companies	investment type is empty?
Opus 5 xhigh	2026-06-30	1,439	593	0
Opus 5 xhigh	2025-12-31	1,408	580	0
Opus 5 medium	2026-06-30	1,439	605	8
Opus 5 medium	2025-12-31	1,409	1,409	1,409
Sonnet 5 medium	2026-06-30	1,439	593	0
Sonnet 5 medium	2025-12-31	1,409	580	1

Why does Opus 5 medium have 1,409 distinct companies, but all of them have an empty investment type field?

Every row became its own borrower

Opus 5 medium, at 2025-12-31:

issuer	investment_type
Automotive Keys Group, LLC and Automotive Keys Investor, LLC, Class A common units	-
Automotive Keys Group, LLC and Automotive Keys Investor, LLC, First lien senior secured loan	-

Opus 5 xhigh, the same rows:

portfolio_company	investment_type
Automotive Keys Group, LLC and Automotive Keys Investor, LLC	Class A common units
Automotive Keys Group, LLC and Automotive Keys Investor, LLC	First lien senior secured loan

- ▶ The problem: the instrument type text has been merged into the borrower name. It parsed the current period correctly and the comparative period wrongly.
- ▶ **And it ties out to the balance sheet exactly, on both dates.** The check was measuring **dollars** while the thing that broke was **structure**.

The broken one is Opus. Sonnet at the same effort is clean.

- ▶ **Sonnet 5 at medium effort is clean:** 593 and 580 distinct companies.
- ▶ Model tier and reasoning effort **did not predict correctness** here.
- ▶ What the extra effort bought was **coverage**, not correctness: 44 columns vs 26, 14 decoded footnote flags, a subtotals file, a legend file.

“Use the biggest model” is not the lesson. How to design verification is the one.

Step 1. Write a spec, then build

Step	What we build	The concept it forces
0	Naive baseline: one prompt	A passing check is only as good as its scope
1	Write a spec, then build	Plan mode, sub-agents for independent verification
2	Generalize across quarters	Skills , and the tie-out as a test suite
3	Analysis and report	Analysis as verification; grounding in your references

- ▶ Now let's do it together.
- ▶ You are going to write one short file and the agent writes everything else.
- ▶ By the end you have a parser, and a test suite that proves it.

The fix is not a longer prompt

- ▶ So how to instruct the agents to make it produce more accurate output?
- ▶ Step 0: one line of instruction, three runs, three schemas. All three reported passing their own checks.
- ▶ The instinct is to **write more instruction**. That leaves you exactly where step 0 left you. Asking the agent to “parse it carefully” has no failing state.
- ▶ What we need to do is to **put the human input where a machine can enforce it**. For example, “zero nulls in the type field, tie-out within 0.1 percent”

Key idea: write down how you will know it worked before you write down what to build.

LAB #1: Write down how you will know it worked, first

Create an empty project folder and write your first document, `plan_v0.md`. It answers one question: **what must be true of the output for me to believe it?**

- ▶ Objective and scope
- ▶ Deliverables and folder structure
- ▶ **Acceptance criteria**

```
1 # BDC Dataset
2
3 * Goal: build a private credit BDC dataset from SEC filings for the most recent filing of one large BDC.
4 * Deliverable:
5   * BDC-quarter panel: one row per BDC per quarter, fund-level financial statements.
6   * BDC-quarter-investment panel: one row per investment position per quarter, deal-level terms.
7   * Two consistent panel schemas.
8 * Acceptance criteria:
9   * BDC-quarter panel:
10     * Each row must be a BDC-quarter observation.
11     * These fields are never null: BDC, CIK, period end, filing date.
12     * total liabilities + net assets = total assets, or other basic accounting rules.
13   * BDC-quarter-investment panel:
14     * Each row must be an investment position.
15     * These fields are never null: BDC, period end, borrower, investment type, amount, fair value.
16     * The sum of extracted fair value equals total investment in the filing, within 0.1%.
17     * Unique borrower names should be fewer than rows, because a BDC holds multiple loans per borrowers.
18   * The units should be consistent across rows and tables.
19   * If any check fails, exit non-zero and write no output file.
20 * Folder structure
21   * README.md
22   * code: filename should be in order, such as "bdc_01_XXX.py", "bdc_09_utils.py", etc.
23   * data:
24     * raw: organized raw files.
25     * interim: temporary processing files.
26   * output: final output.
27   * note: other misc notes.
```

It is much shorter than you expect. Mine is 26 lines.

Example of plan_v0.md

Objective and scope:

```
* Goal: build a private credit BDC dataset from SEC filings for the most recent filing of  
    one large BDC.
```

Deliverables:

```
* Deliverable:  
    * BDC-quarter panel: one row per BDC per quarter, fund-level financial statements.  
    * BDC-quarter-investment panel: one row per investment position per quarter,  
      deal-level terms.  
    * Two consistent panel schemas.
```

The last line is doing quiet work: fixing the incompatible-dataset problem.

Example of plan_v0.md

Folder structure: because I want to be able to find things in three months.

```
* Folder structure
  * README.md
  * code: filename should be in order, such as "bdc_01_XXX.py", "bdc_09_utils.py", etc.
  * data:
    * raw: organized raw files.
    * interim: temporary processing files.
  * output: final output.
  * note: other misc notes.
```

Example of plan_v0.md

Acceptance criteria: this is the machine-checkable statement.

- * BDC-quarter panel:
 - * Each row must be a BDC-quarter observation.
 - * These fields are never null: BDC, CIK, period end, filing date.
 - * $\text{total liabilities} + \text{net assets} = \text{total assets}$, or other basic accounting rules.
- * BDC-quarter-investment panel:
 - * Each row must be an investment position.
 - * These fields are never null: BDC, period end, borrower, investment type, amount, fair value.
 - * The sum of extracted fair value equals total investment in the filing, within 0.1%.
 - * Unique borrower names should be fewer than rows, because a BDC holds multiple loans per borrowers.
- * The units should be consistent across rows and tables.
- * If any check fails, exit non-zero and write no output file.

LAB #2: The prompt

You write `plan_v0.md` yourself – **6 minutes**, under 30 lines:

- ▶ **Objective and scope:** which BDC, which filing, one line.
- ▶ **Deliverables and folder structure:** two panels, one consistent schema.
- ▶ **Acceptance criteria:** which fields are never null; tie-out within 0.1%; borrowers fewer than rows; on any failure, exit non-zero and write no output.

Then prompt the agent:

```
Only work on current folder. Based on @plan_v0.md, propose a working plan in @plan_v1.md.
```

- ▶ Only work on current folder bounds the working directory.
- ▶ `propose...in @plan_v1.md` makes the plan a **file we can edit**, not a message we skim.

26 lines in. 210 lines out

BDC Dataset - Working Plan (v1)

Expands plan_v0.md into an executable build plan. Scope: one BDC, one filing (most recent 10-Q or 10-K), two panels, hard-gated verification.

1. Decisions to lock before coding

Item	Proposed default	Why
Target BDC	ARCC (Ares Capital Corp)	Largest listed BDC, longest filing history, cleanest Schedule of Investments (SOI) layout
Filing	Most recent 10-Q; fall back to 10-K if the latest period end is annual	Latest SOI plus a balance sheet in the same document
CIK resolution	Look up at runtime from https://www.sec.gov/files/company_tickers.json	Do not hardcode a CIK that has not been verified
Language / stack	Python 3.11+, requests, lxml, pandas, pytest	No credentials, no paid data
Units	All money fields stored in USD (float, dollars), not thousands or millions	Filings mix thousands and millions across tables; normalize once at parse time and record the source scale
Output format	CSV plus a Parquet twin; a JSON run manifest	CSV for inspection, Parquet for types

Open question for the user: single BDC only, or should the code be written so the BDC is a parameter from day one? The plan below assumes **parameterized, run for one BDC** (same effort, no lock-in).

- ▶ It came back with 2 schemas, 16 verification checks, 12 named traps, a 7-step build order, and a table of decisions to lock before coding.
- ▶ Because the plan is a file, you can edit one line of it and hand it back.

The plan gave verification gates, and it pushed back on my spec

- `plan_v1.md` written. Three things it adds beyond `v0`:

- **A verification gate that can actually catch a wrong parse.** `v0`'s internal checks (assets = liabilities + net assets, fair value tie-out) all pass if the whole table is off by a factor of 1000. `v1` adds an independent cross-check against EDGAR's XBRL `companyfacts` endpoint, plus an explicit `source_scale` field, so scale errors fail instead of tying out.

- **Named traps with handlers and fixtures.** Ten of them (SOI subtotal rows, split tables, footnote-glued numbers, inherited borrower names, equity rows with no principal). The subtotal-row trap is the likely cause of the first tie-out failure, and the build order expects step 3 to fail before step 4 fixes it.

- **One schema relaxation.** `v0` requires `amount` never-null on the investment panel; equity and warrant positions have no principal. `v1` renames it `principal_amount` and scopes the rule to debt-type rows.

Two open items for you:

Item	Default assumed	Change it?
Target BDC	ARCC	Any large listed BDC works
Parameterization	BDC is a runtime parameter, run once for one BDC	Could hardcode, same effort

CIK is resolved at runtime from `company_tickers.json` rather than hardcoded, since I have not verified any CIK against the live endpoint.

The plan file is where a disagreement between you and the agent becomes visible and cheap. After the code is written, that same disagreement is expensive.

LAB #3: Run it with sub-agents

```
Execute @plan_v1.md. Sub-agent A writes the parser; sub-agent B writes the test from the  
criteria, never seeing the parser.
```

- ▶ A sub-agent is a **fresh context** given one task. It does not see the main conversation, and it disappears when the task is done.
- ▶ That is what makes B's test independent – but only because the criteria live in a file. If they lived in the chat, B would be reading A's homework.

Never let the thing that wrote the answer also write the check on the answer.

Run it with sub-agents

- Step 1 complete: ARCC 10-Q for period 2026-06-30, 24.8 MB, cached with SHA-256. Now launching A and B concurrently and blind to each other.
- 2 background agents launched (↓ to manage)
 - └ Sub-agent A: parser
 - └ Sub-agent B: blind test suite

I will report back when they finish.

* Waiting for 2 background agents to finish

> Show me the tie-out delta when A finishes

Opus 5 (1M context) ctx: [██████████]8% 5h: [██████████]49% @08/16-15:10 7d: [██████████]29% @08/21-07:00

▶▶ auto mode on (shift+tab to cycle) · ← for agents

- main

- general-purpose Inspecting SOI table 9 row structure

2m 2s · ↓ 61.4k tokens

- general-purpose Fetching live companyconcept Assets endpoint

1m 34s · ↓ 45.4k tokens

Sub-Agent A: the parser

Claude Code v2.1.233

Welcome back Xugan!



Opus 5 (1M context) with medi... · Claude Max ·

Tips for getting started

Run `/init` to create a `CLAUDE.md` file with instructions...

What's new

Added GitLab merge request URL support to the `--workt...`
Added an opt-in `'forward_user_identity'` apps gateway s...
Added opt-in memory cgroup support for Bash tool comma...
/release-notes for more

▲ 1 MCP server needs authentication · run `/mcp`

> You are sub-agent A on a two-agent blind build. You write the PARSER. Another agent is independently writing the test suite from the written criteria without seeing your code. You must NOT write, read, or modify anything under `'tests/'`, and you must NOT create or edit `'code/bdc_08_checks.py'`. Do not try to discover what the tests assert. Write to the spec, not to the test.

PROJECT ROOT (work only here):

READ FIRST: `'plan_v1.md'` in that root. It is the contract. Sections 3 (schemas), 5 (traps), 6 (folder

Sub-agent A: parser

>

Opus 5 (1M context) ctx: [] 8% 5h: [] 49% @08/16-15:10 7d: [] 29% @08/21-07:00
Enter to view · x to stop · ctrl+x ctrl+k to stop all agents

○ main

> ● general-purpose Scanning SOI tables for subtotal rows 2m 34s · ↓ 65.1k tokens

○ general-purpose Extracting inline XBRL from arcc-20260630.htm 2m 6s · ↓ 47.7k tokens

Sub-Agent B: the test

Claude Code v2.1.233

Welcome back Xugan!



Opus 5 (1M context) with medi... · Claude Max ·

Tips for getting started

Run `/init` to create a `CLAUDE.md` file with instructions...

What's new

Added GitLab merge request URL support to the `--workt...`
Added an opt-in `forward_user_identity` apps gateway s...
Added opt-in memory cgroup support for Bash tool comma...
/release-notes for more

▲ 1 MCP server needs authentication · run `/mcp`

› You are sub-agent B on a two-agent blind build. You write the VERIFICATION GATE and the TEST SUITE, working ONLY from the written criteria and from the source filing. Another agent is independently writing the parser right now.

HARD RULE, this is the point of the exercise: you must NEVER read, open, grep, list, or infer the contents of the parser. Specifically off-limits: ``code/bdc_03_extract.py``, ``code/bdc_04_parse_soi.py``, ``code/bdc_05_parse_bs.py``, ``code/bdc_06_normalize.py``, ``code/bdc_07_panels.py``, ``code/run_all.py``, and any file under ``data/interim/`` that the parser produces (``soi_rows.csv``, ``balance_sheet.json``, the panel CSVs). If those files exist, do not look at them. You are allowed to read ``code/bdc_09_utils.py``, ``code/bdc_01_resolve.py``, ``code/bdc_02_fetch.py``, ``data/interim/target_filing.json``, and

Sub-agent B: blind test suite

›

Opus 5 (1M context) ctx: [] 8% 5h: [] 49% @08/16-15:10 7d: [] 29% @08/21-07:00
Enter to view · x to stop · ctrl+x ctrl+k to stop all agents

○ main

○ general-purpose Summing SOI fair values against balance-sheet total

2m 46s · ↓ 66.3k tokens

› ● general-purpose Counting ix:nonFraction tag names

2m 18s · ↓ 49.5k tokens

Sub-Agent B: the test

● Agent "Sub-agent B: blind test suite" finished · 14m 44s

- Sub-agent B has finished. Sub-agent A is still running, so these numbers are B's report and my own independent verification is pending.

B confirmed it never read the parser. It touched parser output only through `pandas.read_csv` on the two panel CSVs, which is what the gate is supposed to do.

Ground truth, derived three independent ways (rendered balance-sheet text, the filing's own inline XBRL facts, and the companyfacts API):

Field	2026-06-30	2025-12-31
Total investments at FV	29,349,000,000	29,485,000,000
Total assets	30,498,000,000	31,235,000,000
Total liabilities	16,607,000,000	16,917,000,000
Net assets	13,891,000,000	14,318,000,000
NAV per share	19.35	19.94

The filing reports in millions, confirming the per-table scale concern was real. The comparative date was read from the section heading and cross-confirmed by inline XBRL contexts, not inferred.

What it bought, and what it did not

▶ What it bought:

- ▶ A **structured and staged pipeline**: 2,757 lines, 10 scripts
- ▶ An **independent test suite**: 1,067 lines, reusable next quarter
- ▶ Criteria that catch the step 0 break: type never null, borrowers fewer than rows
- ▶ A gate that writes nothing on failure

⇒ Now we have the confidence to know the numbers are right.

▶ What it did not buy:

- ▶ A better answer: the output is almost identical to step 0's
- ▶ Speed: 28 min against 15 to 23 min

This works for one filing. There are 88.

- ▶ You now have a parser that foots to the balance sheet, plus a test suite that proves it, on **one** 10-Q.
- ▶ ARCC has **88 filings** on EDGAR, back to 2004.
- ▶ Run this parser, unchanged, across all 88: only 5 filings work.
- ▶ But we don't want to write the other 83 parsers manually—so we need to **generalize**.

Step 2. Generalize across quarters

Step	What we build	The concept it forces
0	Naive baseline: one prompt	A passing check is only as good as its scope
1	Write a spec, then build	Plan mode, sub-agents for independent verification
2	Generalize across quarters	Skills, and the tie-out as a test suite
3	Analysis and report	Analysis as verification; grounding in your references

- ▶ The parser in step 1 works on **one** filing. A dataset is many.
 - ▶ Today: generalize across quarters.
 - ▶ Another dimension: generalize across BDCs.
- ▶ We are going to package what we learned as a **skill**, then sweep the whole history.
- ▶ The deliverable of this step also includes a **coverage statement** that says what the panel does and does not contain.

A script computes. A skill carries the judgment.

- ▶ Wrong definition: “a script has the implementation, a skill has the prose.” Follow it and you rewrite 2,757 working lines for every new filer.
- ▶ Right definition:

A skill is the map of (1) what is reusable, (2) what is not, and (3) how to tell the difference on an input it has not seen.

- ▶ It **points at** the pipeline and carries the boundary – the thing that does not survive in code and has to be generalized.
- ▶ Plus: how to detect which representation a filing uses, the acceptance criteria, the traps with their signatures, and what it must **never** do to make a check pass.

LAB #4: Prompt: package the seam, with a repair loop

Read code/, note/trap_log.md and note/parsing_decisions.md.

First, tell me which parts of this pipeline are filer-agnostic and which encode something specific to this BDC. Quote the lines.

Then write .claude/skills/bdc-soi-parse/SKILL.md so that an agent facing a BDC or a quarter we have never parsed knows: what to reuse unchanged, what to extend and how, how to detect which representation a filing uses, the acceptance criteria, and each trap with the signature that detects it. It must also say how to repair itself, and what it must never do in order to make a check pass.

Do not paste the engine into the skill. Point at it.

The skill

- Written to `.claude/skills/bdc-soi-parse/SKILL.md` (255 lines). Structure:

Section	Contents
1	Reuse-unchanged table: 12 mechanisms with <code>file:line</code> , no code pasted
2	Extend-here table: 15 shared-config tables mapped to the symptom that sends you there, plus the two structural extensions (borrower-on-own-row, control-status sectioning)
3	Representation detection: 15 probes keyed to <code>extract.json</code> / <code>soi_rows.csv</code> fields
4	<code>plan_v0.md</code> acceptance criteria verbatim, the one documented relaxation, and which of the 16 checks matter most on an unfamiliar filing
5	All 12 traps plus the two real bugs, each with a detection signature, handler location, and whether ARCC exercised it
6	Self-repair: 6-step failure protocol, an 11-item never-do list, and the trap-log append template
7	Command sequence for orienting on a new filing

Two things I flagged rather than smoothed over:

- **Trap 4 (parenthesised negatives) is untested against real data.** The handler exists; the ARCC filing never exercised it. The skill marks it and names it as the likely first entry to move to "fired".
- **A silent-wrong-but-green path exists:** if the non-accrual footnote legend is not found, `bdc_03_extract.py:336` logs a warning, `is_non_accrual` is False for every row, and no check catches it. The skill calls this out in the detection table.

LAB #5: Prompt: parameterize, then sweep

/bdc-soi-parse ARCC prior 10-Q (2026-03-31)

- ▶ The skill carries everything we learned in step 1 and transferred to other quarters.

- ARCC 10-Q, period end 2026-03-31, accession 0001628280-26-027688: 16 pass, 0 fail, 0 warn, 0 skip, promoted to output/.
 - No parser, vocabulary, or tolerance change was needed. Every shared table in the skill's section 2 already matched: SOI headers, balance-sheet labels, type vocabulary, rate canon, scale words, XBRL tags.
 - One extension, and it was a selection gap, not a representation difference: the pipeline could not target a non-latest filing at all.
 - Tie-out 0.001017% (tolerance 0.1%); against the SOI's own printed grand total, 0.000000%.

The extension

`pick_filing` sorted by period end and returned `rows[0]`, so only the newest 10-Q/10-K was reachable. Added to the shared resolver, serving every filer and quarter:

Change	File
<code>period_end / form</code> selectors, exposed as <code>--period-end / --form</code>	<code>code/bdc_01_resolve.py:26-53</code>
<code>main(argv=None)</code>	<code>code/bdc_01_resolve.py:86</code>
Selectors forwarded to stage 01 only; gate called as <code>main([])</code> so its <code>argparse</code> never sees them	<code>code/run_all.py:41-49,78,95-112</code>

Exact selector, not nearest-match: an unreported period fails with the list of periods that exist; two filings sharing a period end fail asking for `--form`. No accession, CIK, or index hardcoded, and the no-selector default is unchanged.

LAB #6: Prompt: parameterize, then sweep

```
/bdc-soi-parse every 10-K and 10-Q ARCC has filed with a period end on or after 2023-01-01. Work  
oldest to newest so failures cluster visibly.
```

Then assemble all successful filings into a single BDC-quarter panel and check that (cik,
period_end) is unique. Report what you find before fixing it.

- ▶ I ran it three times; each time it found new failures and fixed some of them.
- ▶ Pipeline grew 2,757 to 4,009 lines (+45%); `trap_log.md` reached 491 lines.
- ▶ 37,197 positions; **30 of 34** from 2018 onward; **14 of 14** from 2023 onward.

Pass	In panel	Note
Baseline (step 1 parser, unchanged)	5 of 88	
After first sweep and repair	22 of 88	Went backwards in time, oldest first
After another pass, asking it to try again	43 of 88	Reached 2005; the recent hole untouched
Focused on post-2018, newest first	50 of 88	The recent hole closed

What you already trust must not move

Reference filing, 2026-06-30	After step 1	After step 2
Positions	1,439	1,439
Distinct borrowers	593	593
Sum of fair value (USD)	29,349,300,000	29,349,300,000

- ▶ Byte-identical through **45 filings** of edits.
- ▶ It kept a saved copy and diffed after every change.
- ▶ It also recorded a bug it **introduced** during the backfill.

When you extend a model, make sure the thing you already trust does not move.

Step 3. Analysis and report

Step	What we build	The concept it forces
0	Naive baseline: one prompt	A passing check is only as good as its scope
1	Write a spec, then build	Plan mode, sub-agents for independent verification
2	Generalize across quarters	Skills , and the tie-out as a test suite
3	Analysis and report	Analysis as verification; grounding in your references

- ▶ After Step 2, we have a panel ready for analysis.
- ▶ One prompt to analyze the panel and produce a report.
- ▶ And using the data turns out to be **a check of its own**.

LAB #7: The prompt

ultracode

```
Based on the 2018-2026 data in output/panel/bdc_quarter.csv and
    output/panel/bdc_quarter_investment.csv, produce an industry report on this portfolio. Write
    the code to analysis and output everything to output/analysis. The analysis report style
    follows the documents in reference, and should be in both .docx and .pdf, with figures and
    tables. It should at least include sample coverage, summary statistics, time series, deal
    terms, and cross-section. Be specific about percentages and periods. Every figure and table
    states the period it covers and the share of rows it is computed on.
```

Ultracode

- ▶ **Ultracode** is dynamic workflows orchestrate many subagents, skills, and tools, so that it can plan and execute complex tasks.
- ▶ Same mechanism as the sub-agents in step 1, at a larger scale and scheduled by the tool rather than by me.

```
● Workflow(Build the ARCC 2018-2026 private-credit portfolio industry report: analysis modules, figures, tables, then a synthesized report)
  └ Running in background · /workflows to monitor and save · medium size (/config)
```

```
● While that runs, I'll prepare and test the d Jump to bottom (click) ↓
```

arcc-portfolio-report

Build the ARCC 2018-2026 private-credit portfolio industry report: analysis modules, figure...3/8 agents · 7m08s

Phases

```
> 1 Analyze 3/5
  2 Verify 0/3
  3 Write
```

Analyze · 5 agents

✓ analyze:coverage	Opus 5 (1M context) · 89.3k tok	6m48s
✓ analyze:summary	Opus 5 (1M context) · 76.5k tok	5m14s
✓ analyze:timeseries	Opus 5 (1M context) · 83.3k tok	5m15s
● analyze:terms	Opus 5 (1M context) · 103k tok	7m06s
● analyze:crosssection	Opus 5 (1M context) · 98.9k tok	7m06s

Ultracode

- ▶ **Ultracode** is dynamic workflows orchestrate many subagents, skills, and tools, so that it can plan and execute complex tasks.
- ▶ Same mechanism as the sub-agents in step 1, at a larger scale and scheduled by the tool rather than by me.

```
● Workflow(Build the ARCC 2018-2026 private-credit portfolio industry report: analysis modules, figures, tables, then a synthesized report)  
└ Running in background · /workflows to monitor and save · medium size (/config)
```

- While that runs, I'll prepare and test the d Jump to bottom (click) ↓

arcc-portfolio-report

Build the ARCC 2018-2026 private-credit portfolio industry report: analysis modules, figure...3/8 agents · 7m14s

Phases

```
1 Analyze 3/5  
2 Verify 0/3  
3 Write
```

Verify · 3 agents

● verify:summary	Opus 5 (1M context) · 60.6k tok	1m52s
● verify:timeseries	Opus 5 (1M context) · 63.9k tok	1m50s
● verify:coverage	Opus 5 (1M context) · 31.6k tok	19s

Style in `reference/`

- ▶ Prompt: “**the analysis report style follows the documents in reference**”.
- ▶ We want to control the style and format of the report. It can look like an industry report, or an academic paper. But we don't want to type every detail by hand.
- ▶ `reference/` holds three industry report PDFs I downloaded.
- ▶ **What transferred:** exhibit numbering, a source line under every exhibit, a “Finding.” sentence opening each section, a limitations section.

Further usage

- ▶ **Build an index** once the corpus is too big to attach every time: queried rather than pasted, and reusable. Part 4.
- ▶ **Turn your own style into a skill**, so the next report starts from your conventions.

One prompt, 53 pages

- Verified and complete. Every executive-summary number reproduces exactly against the raw panels on my own independent recomputation.

Deliverables

File	Size
output/analysis/ARCC_Private_Credit_Portfolio_Report.pdf	5.3 MB, 53 pages
output/analysis/ARCC_Private_Credit_Portfolio_Report.docx	5.4 MB, 338 paragraphs, 25 tables, 32 embedded images
output/analysis/report.md	source, 175 KB, 0 non-ASCII characters
output/analysis/figures/ tables/ facts/	32 PNG, 24 CSV + 24 MD, 56 JSON fact sidecars
code/analysis/	common.py (scaffold), m1-m5 modules, build_report.py

What the report finds (2018Q3–2026Q2, 30 quarters, 31,067 position-quarter rows, ARCC only)

- Fair value \$11.220bn to \$29.349bn, +161.6% (13.21%/yr over 7.75 yrs); net assets +89.9% (8.63%/yr), so debt/net assets went 0.622x to 1.135x, peak 1.278x in 2022Q4.
- First lien 44.1% to 59.1% of fair value, second lien 29.9% to 4.4%, equity 9.0% to 15.4%. Senior secured combined actually fell, 74.0% to 63.5%.
- All-in rate on the debt book: 7.82% trough 2021Q4, 12.20% peak 2023Q3, 9.49% in 2026Q2, while FV-weighted spread compressed 707bps to 570bps. LIBOR-to-SOFR crossover in a single quarter, 2023Q2.
- Non-accruals 5.06% of cost peak (2020Q3), 2.38% of cost / 1.38% of fair value in 2026Q2; non-accrual debt carried at 0.5839x cost.
- Borrowers 313 to 593; top-10 share 25.55% to 22.43%. Software and Services +9.09pp to 21.98%; Healthcare Equipment and Services -10.42pp to 9.86%.

- Report: 53 pages, 21,006 words, 32 figures and 48 tables.
- Analysis is not the cheap part: 4,086 lines of code. The previous pipeline: 4,009 lines.

The report

Inside a Direct Lender's Book

The Ares Capital Corporation private-credit portfolio reconstructed from SEC Schedules of Investments, 2018Q3-2026Q2

Prepared with Claude Code

2026-08-16

Contents

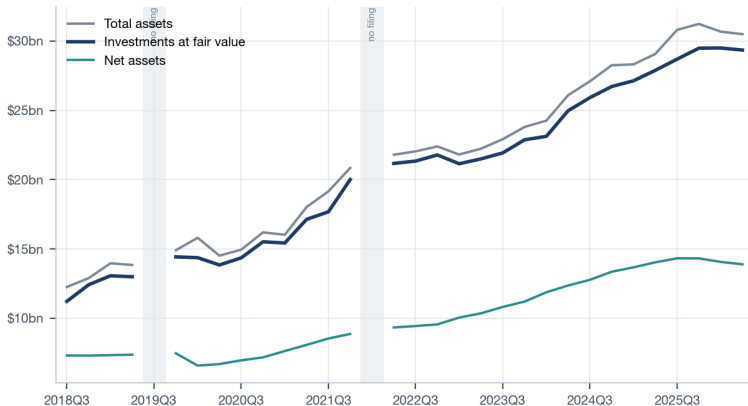
Inside a Direct Lender's Book	2
The Ares Capital Corporation private-credit portfolio, 2018Q3-2026Q2	2
1. Executive summary	2
2. Sample coverage and data quality	3
2.1 Which quarters exist	3
2.2 Field completeness	5
2.3 Missingness is structural, on two axes at once	6
2.4 How much of the book the rate exhibits actually describe	8
2.5 The internal consistency check	10
3. Summary statistics	11
3.1 Scale	11
3.2 Position size and concentration	13
3.3 Marks	16
3.4 Non-accruals	18
3.5 Effective yield	20
4. Time series	22
4.1 Growth	22
4.2 Leverage	23
4.3 Asset mix	25
4.4 Reference-rate migration	26
4.5 Yield and spread	28
4.6 Credit stress	29
5. Deal terms	31
5.1 Spread by seniority	31
5.2 Decomposing the all-in rate	34
5.3 Fixed versus floating	35
5.4 Maturity structure	36
5.5 Do wider spreads carry worse marks?	38
6. Cross-section	39
6.1 Industry rotation	40
6.2 Industry concentration	41
6.3 Borrower breadth, concentration and persistence	42

- ▶ Eight sections, among them: executive summary, coverage, summary statistics, time series, deal terms, cross-section, limitations.
- ▶ Every exhibit carries **its period and its coverage**, because the prompt said so and the code enforced it.

Example exhibit: the time series of balance sheet growth

Balance sheet in levels, quarter by quarter

Investments at fair value grew from \$11.2bn to \$29.3bn (13.2% per year over 7.75 years)



Period covered: 2018Q3-2026Q2. Computed on 30 of 30 quarter-panel rows (100.0% of the window panel). The panel holds 30 of the 32 calendar quarters in the window: 2019Q3 and 2022Q1 are absent and are shown as breaks; no value is interpolated or bridged across them. The growth rate is an endpoint-to-endpoint compound rate over the 7.75 years from 2018-09-30 to 2026-06-30, not an average of quarterly growth.

Source: author's calculations on the ARCC BDC panel built from SEC EDGAR 10-K/10-Q filings.

The analysis found a bug that previous checks passed over

The report, in its own section 4.6, **unprompted**:

*“The non-accrual series is drawn on 27 of the 30 quarters: 2022Q4, 2023Q4 and 2024Q4 return exactly zero parsed flags while neighbouring quarters carry 13 to 22, which is a **footnote-marker parse failure and not a clean book**.”*

- ▶ **Why no check saw it.** `is_non_accrual` is a boolean, so **no tie-out reaches it** – and the column is 0.00% null: fully populated, and in three quarters uniformly wrong.

Analysis can be viewed as another verification layer.

Part 3. More useful concepts

Working tips

1. Plan before acting.
2. Be specific.
3. Always have a backup.
4. Manage your context window.
5. Always have a way to verify outputs.
6. Ask the LLM to take notes.
7. Be replicable.
8. Stay organized with your code, data, and notes.

AI safety: honestly, you will grant most permissions

- ▶ Every step today asked for file writes, shell access and network access. It is unrealistic to approve every single permission every time.
- ▶ The best you can do is to allow-list broadly and stop reading.
- ▶ Which is tip 3 again, for a different reason: **always have a backup**.
- ▶ Mine is GitHub for code and Dropbox for data, and my own agents run on **a separate Mac Mini** – a machine with limited access to my data that I can wipe.
- ▶ The real question is not “what may it do” but **“what can it reach”**.

Dev containers are the real alternative

- ▶ Cost: Docker, a config file, and a slower first run. Windows needs WSL2.
- ▶ Isolation is about **what is reachable**.
- ▶ See “#7 Permissions, Sandboxes, and Autonomous Agents” in *A Series on Claude Code*, by Paul Goldsmith-Pinkham.

Part 4. Advanced usage examples

Each of these adds something the project could not show

Demo	Axis it adds	Where Part 2 stopped
TravelTab + PetTab	The pipeline becomes <i>software</i>	We shipped a report, not a system
GBrain	Retrieval that is <i>indexed</i>	Step 3 attached sample docs by hand
NanoClaw	The agent has an <i>address</i>	You were always at the terminal

Nothing here is a new tool. It is the same agent, moved one step along one axis.

TravelTab: personal-database management

- ▶ Background: I travel between New Haven and Philadelphia constantly, and Amtrak is the only workable way to do it. The app is clunky, it does not talk to my calendar, **the ticket arrives once by email and sometimes it is gone**, and the pricing is dynamic.
- ▶ My old routine: dig each ticket out of my inbox, save it, and copy the trip into my calendar by hand; and often **pay twice what the same seat had cost a week earlier**.
- ▶ So I built **TravelTab**, a personal Amtrak tracker that manages my trips **automatically**:
 1. **Ingests tickets automatically from my email**.
 2. **Exports structured records into a database**: one row per passenger per ticket.
 3. **Syncs into my calendar**, so a trip is something I can actually manage.
 4. **Fare watch** that checks prices and alerts me.
- ▶ Now I just book a trip, and it's done. I can reach it **from anywhere on any device**.

TravelTab: personal-database management

TravelTab Tickets Spending Fares Grid Import Settings Sign out

Tickets

+ New

Search route, train, passenger, confirmation...

All accounts Xugan Chen All routes All statuses

Future Past **All** Date Month Year Clear all

All 806 Confirmed 286 Canceled 520

List Trend 1-100 of 286

RESERVATION	DATE / TIME ↑	TRAIN	ROUTE	PASSENGER	ACCOUNT	PRICE	SAVINGS	STATUS	
		79	PHL-WAS	Xugan Chen		\$38.00	-	Confirmed	✉ 📄 🗑
		90	WAS-NWK	Xugan Chen		\$62.00	-	Confirmed	✉ 📄 🗑
		143	NHV-MET	Xugan Chen		\$17.00	-	Confirmed	✉ 📄 🗑
		143	NHV-PHL	Xugan Chen		\$128.00	-	Confirmed	✉ 📄 🗑
		96	PHL-NHV	Xugan Chen		\$67.15	-	Confirmed	✉ 📄 🗑
		173	NHV-PHL	Xugan Chen		\$68.85	-	Confirmed	✉ 📄 🗑
		56	PHL-NHV	Xugan Chen		\$22.00	-	Confirmed	✉ 📄 🗑
		55	NHV-PHL	Xugan Chen		\$18.70	-	Confirmed	✉ 📄 🗑
		56	PHL-NHV	Xugan Chen		\$18.70	-	Confirmed	✉ 📄 🗑

Every row here arrived as **an email**. Messy documents in, structured records out.

And it keeps running when you are not watching

TravelTab Tickets Spending **Fares** Grid Import Settings winnie Sign out

Fares Fetch fares Discount % Alert if drop ≥ %

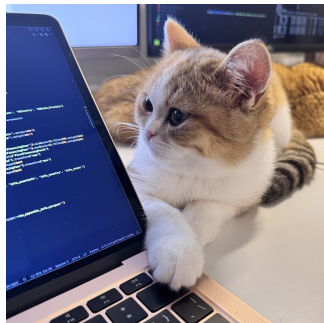
Comparisons use your 15% discount (effective price = listed fare - 15%).

From To Date Train # Label (optional) + Add watch

ROUTE	DATE	TRAIN	BOOKED	LOWEST	LATEST	SAVINGS		QTY	LAST FETCH (ET)	
NHV-PHL		173	\$43.35	\$34.85	\$35.70	+\$7.65	↘ Drop	124	8/17, 9:59 PM	  
NHV-PHL		141	\$23.80	\$23.80	\$23.80	\$0.00		109	8/17, 9:57 PM	  
PHL-NHV		170	\$23.80	\$23.80	\$23.80	\$0.00		190	8/17, 9:57 PM	  
NHV-PHL		141	\$23.80	\$23.80	\$23.80	\$0.00		170	8/17, 9:57 PM	  
PHL-NHV		170	\$23.80	\$23.80	\$23.80	\$0.00		190	8/17, 9:57 PM	  
NHV-PHL		1143	\$23.80	\$23.80	\$23.80	\$0.00		1	8/17, 9:57 PM	  
PHL-NHV		1194	\$45.05	\$45.05	\$45.05	\$0.00		1	8/17, 9:57 PM	  
PHL-NHV		1150	\$23.80	\$23.80	\$23.80	\$0.00		1	8/17, 9:58 PM	  
PHL-NHV		170	\$23.80	\$23.80	\$23.80	\$0.00		190	8/17, 9:58 PM	  
NHV-PHL		141	\$18.70	\$18.70	\$18.70	\$0.00		170	8/17, 9:57 PM	  
NHV-PHL		141	\$18.70	\$18.70	\$18.70	\$0.00		170	8/17, 9:58 PM	  
PHL-NHV		170	\$23.80	\$23.80	\$23.80	\$0.00		190	8/17, 9:57 PM	  

Fare watchlist that checks prices on a schedule and alerts me on a threshold.

PetTab: another domain, same stack

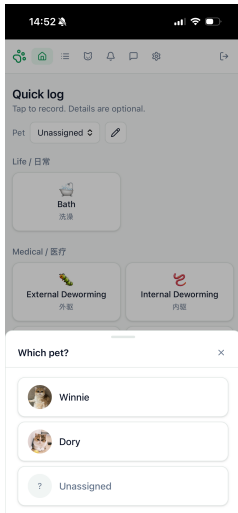


- ▶ Background: I have two cats, Winnie and Dory, and I want to track their health and behaviour. Existing tools are too complicated, and I want full control over the data.
- ▶ So I built **PetTab**, a pet-health logger built on TravelTab's stack.

PetTab: easy to use in 3 clicks



1. Tap what happened.

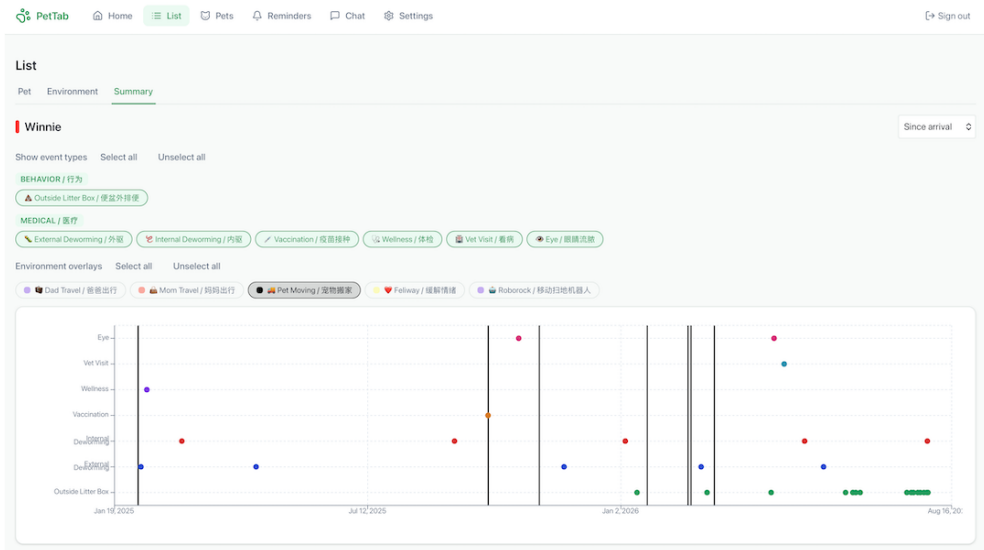


2. Tap which cat.



3. Logged, and editable.

PetTab: visualizing the data



PetTab: a chat agent inside the product

The screenshot shows the PetTab web application. The top navigation bar includes links for Home, List, Pets, Reminders, Chat (highlighted), and Settings, along with a Sign out button. The left sidebar shows a chat history with three entries: 'Winnie and Dory Vaccination...' (Aug 19, 2026), 'Winnie and Dory Vaccination...' (Aug 19, 2026), and 'Winnie and Dory Behavior...' (Aug 18, 2026). The main chat area displays a message: 'When did Winnie and Dory receive their vaccinations, and when is the next one due?'. Below this is a 'Vaccination History' table:

Pet	Date	Vaccine(s) given (per notes)
Dory	2025-07-22	FelV Vaccine (booster in 3-4 weeks) · FVRCP Vaccine (1 Year)
Dory	2025-08-12	FelV Vaccine (1 Year) — this was the booster
Winnie	2025-10-02	Rabies Vaccine (Recombinant, 1 Year) · FVRCP Vaccine (1 Year)

Below the table, it says 'Next due dates'. A note explains that there are no active reminders set up, so PetTab is inferring next due dates based on the '1 Year' interval in the vaccination notes. A second table shows the inferred next due dates:

Pet	Vaccine	Inferred next due
Dory	FVRCP	~2026-07-22
Dory	FelV	~2026-08-12
Winnie	Rabies	~2026-10-02
Winnie	FVRCP	~2026-10-02

A caveat note states: 'Caveat: these dates are inferred from free-text in the event notes ("1 Year"), not from any structured due-date field — no reminder records exist for these vaccines to confirm them.' At the bottom, there is a text input field with the placeholder 'Ask about your pet data...' and a green send button.

- ▶ An agent is embedded within the product.
- ▶ PetTab's Insights chat runs the `claude` CLI over a **read-only snapshot** of the database, so that it can explore the database and give me suggestions or help me manage the reminders.

GBrain: personal-knowledge brain

You ask:

"What do I need to know before my meeting with Alice tomorrow?"

Most personal-knowledge tools give you back a list of pages. Something like:

1. people/alice - Alice runs engineering at Acme...
2. meetings/2026-03-15-alice-q1 - Q1 product review with Alice...
3. meetings/2026-01-08-acme-kickoff - Kickoff meeting with Acme team
4. customers/acme - Acme is a series-B fintech we work with...
5. notes/2026-04-22 - Quick chat with Alice about pricing...

GBrain gives you back the answer, with sources:

Alice runs engineering at Acme (a series-B fintech). You last spoke on April 22 in a quick pricing chat. Three things are still open from that conversation:

1. She owes you the security review for the new tier (deadline was May 1; no update since).
2. You committed to pricing for a 500-seat tier (you sent it April 25; no response yet).
3. She mentioned they're hiring a CISO; you said you'd intro someone from your network.

Heads up: nothing's been added to the brain about Alice or Acme since April 22, six weeks ago. She may have replied through email or Slack DM, channels the brain doesn't see. Worth asking her to catch up before assuming any of this is still current.

- ▶ **GBrain** indexes a corpus you own – meeting notes, summaries, Notion – and exposes it to the agent as a tool it can query.
- ▶ The key idea: it (1) searches the corpus, (2) reads the hits for you, and (3) writes the answer.
- ▶ github.com/garrytan/gbrain

NanoClaw: an agent with an address



NanoClaw

An AI assistant that runs agents securely in their own containers. Lightweight, built to be easily understood and completely customized for your needs.

nanoclave.dev • [docs](#) • [中文](#) • [日本語](#) • [한국어](#) •  Discord 519 online •  tokens 226k

Why I Built NanoClaw

[OpenClaw](#) is an impressive project, but I wouldn't have been able to sleep if I had given complex software I didn't understand full access to my life. OpenClaw has nearly half a million lines of code, 53 config files, and 70+ dependencies. Its security is at the application level (allowlists, pairing codes) rather than true OS-level isolation. Everything runs in one Node process with shared memory.

NanoClaw provides that same core functionality, but in a codebase small enough to understand: one process and a handful of files. Agents run in their own Linux containers with filesystem isolation, not merely behind permission checks.

- ▶ **NanoClaw**: Claude in a Linux container, reached from [WhatsApp](#), [Slack](#), [Telegram](#) or email.
- ▶ Every minute so far had you at a terminal, driving. Same tool – reached by sending it a message.
- ▶ A personal-scale project: one process and a handful of files, running in a Linux container.
- ▶ github.com/nanocoai/nanoclave

Where to go next

- ▶ Try to use it in your own projects and daily life **as much as possible**.
- ▶ Brainstorm your ideas, and then **build them**.

Other useful resources:

- ▶ [Agent Skills with Anthropic](#) – DeepLearning.AI short course
- ▶ [Finance skill plugins](#) by Anthropic
- ▶ A collection of resources on AI tools: www.xuganchen.com/public-goods

Course materials and every prompt run today:

github.com/xuganchen/BootCamp-Introduction-to-Claude-Code